

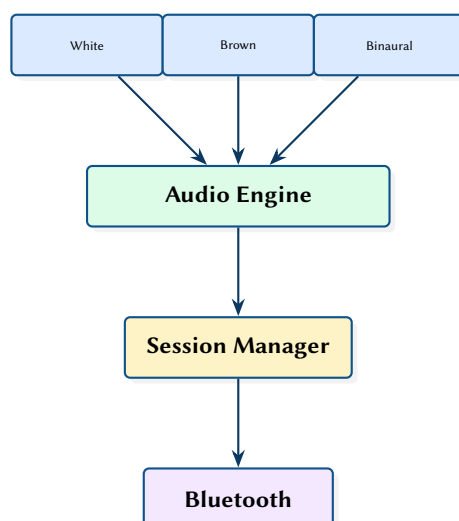
WatchNoise: Real-Time Audio Synthesis on Apple Watch

Noise Generation, Binaural Beats, and Bluetooth-Aware Playback

Simon Knight

March 2026 • Version 0.1.0

Last updated: March 12, 2026



Abstract. WatchNoise is a minimalist Apple Watch application that generates ambient noise for sleep through Bluetooth headphones. The app synthesises white noise, brown noise (IIR low-pass filtered), smooth time-varying blends, and binaural beats at configurable frequencies using AVAudioSourceNode for real-time procedural generation. A four-state playback machine manages audio session lifecycle, Bluetooth routing, and automatic pause on headphone disconnect. This report covers the audio synthesis algorithms, stereo rendering architecture, and watchOS integration constraints.

Contents

1	Introduction and Motivation	2
2	Audio Synthesis	2
2.1	White Noise	2
2.2	Brown Noise	2
2.3	Blended Noise	3
2.4	Binaural Beats	3
3	Playback State Machine	4
4	Design Summary	5
	List of Figures	6
	List of Tables	6
	List of Design Decisions & Rules	7
	Revision History	7
IIR	Infinite Impulse Response	2

1 Introduction and Motivation

Many people use ambient noise to fall asleep, but existing phone-based apps drain the phone battery and require keeping the phone nearby. WatchNoise runs entirely on Apple Watch, streaming audio directly to Bluetooth headphones from the wrist.

The app synthesises audio procedurally rather than playing pre-recorded loops, avoiding the perceptible repeat point that can disturb light sleepers.

: Procedural Generation Only

All audio is generated in real-time using mathematical functions. No audio files are bundled with the app. This eliminates storage overhead on the watch and provides infinite non-repeating output.

2 Audio Synthesis

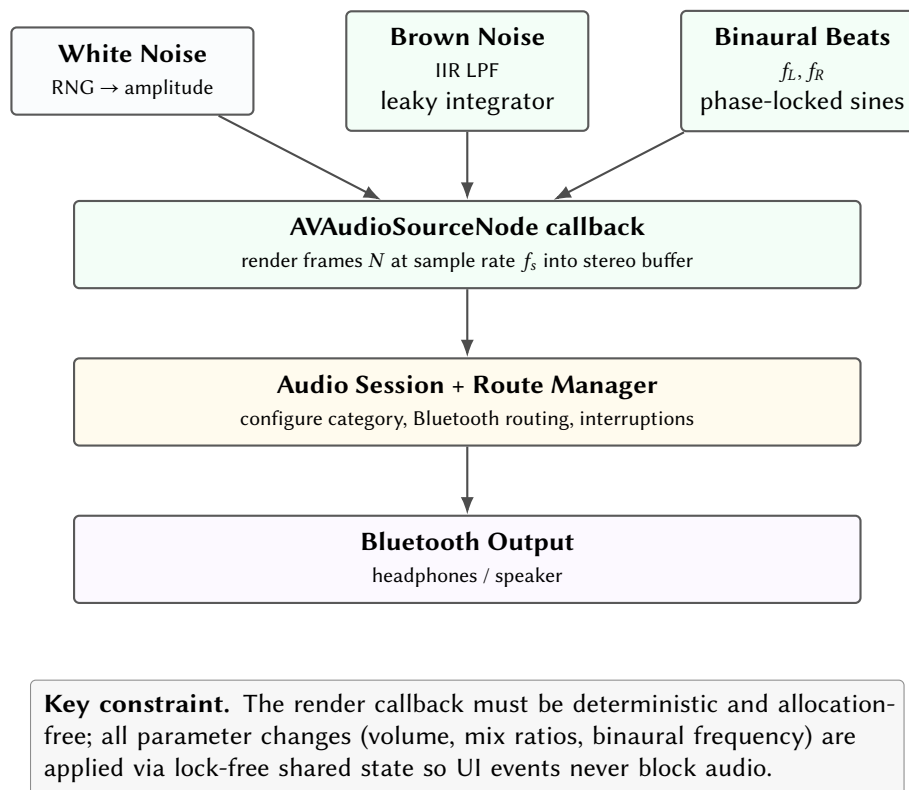


Figure 1. WatchNoise audio synthesis pipeline. The app is built around the real-time render callback; everything else exists to feed it parameters safely.

2.1 White Noise

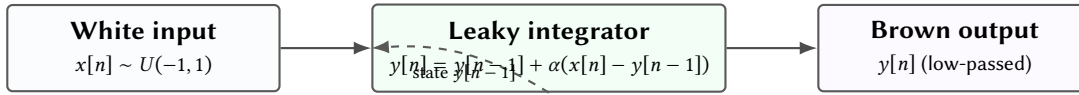
White noise is generated by sampling from a uniform random distribution and scaling to the desired amplitude:

```
func generate(into buffer: AVAudioPCMBuffer) {
    let frames = Int(buffer.frameLength)
    for i in 0..
```

2.2 Brown Noise

Brown noise (red noise) is produced by applying a first-order Infinite Impulse Response (IIR) low-pass filter to white noise, accumulating samples with a leaky integrator:

Brown noise as a 1st-order IIR: shaping spectrum with one state variable



Conceptual point. Brown noise is not "a different RNG"; it is white noise passed through a 1-pole low-pass. The single state $y[n-1]$ is why it is cheap enough for watchOS real-time audio.

Figure 2. Brown noise generation as a minimal-state filter. One multiply-add plus one state variable shapes the spectrum toward low frequencies.

```
func generate(into buffer: AVAudioPCMBuffer) {
    let frames = Int(buffer.frameLength)
    let alpha: Float = 0.02 // Filter coefficient
    for i in 0..

```

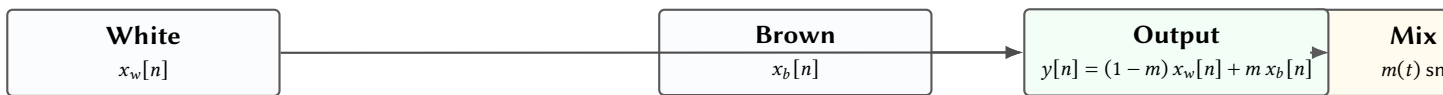
The gain factor of 10.0 compensates for the low-pass filter's attenuation. The filter coefficient $\alpha = 0.02$ produces a -3 dB point at approximately:

$$f_{-3\text{dB}} = \frac{f_s}{2\pi} \cdot \alpha \approx \frac{44100}{6.28} \cdot 0.02 \approx 140 \text{ Hz} \quad (1)$$

2.3 Blended Noise

The blended generator produces a time-varying mix of white and brown noise, smoothly cross-fading over a configurable period to create a gently evolving soundscape.

Blending as a control signal: crossfade without clicks



Conceptual point. Click-free blending requires the mix coefficient change slowly relative to the audio sample rate. Treat the UI plane: it sets targets; the render loop applies a smoothed coefficient.

Figure 3. Noise blending as control-signal composition. The UI sets a target mix ratio; the audio thread applies a smoothed crossfade per sample (or per block) to avoid discontinuities.

2.4 Binaural Beats

Binaural beats [1] create the perception of a third tone by playing slightly different frequencies in each ear. The beat frequency equals the difference between left and right:

$$f_{\text{beat}} = |f_L - f_R| \quad (2)$$

Table 1. Binaural beat presets.

Preset	Beat Freq	Base Freq	Brain Wave
Deep Sleep	2 Hz	200 Hz	Delta
Sleep	4 Hz	200 Hz	Delta
Meditation	6 Hz	250 Hz	Theta
Relaxation	8 Hz	250 Hz	Alpha
Focus	15 Hz	300 Hz	Beta

The stereo generator renders sine waves at f_{base} (left) and $f_{\text{base}} + f_{\text{beat}}$ (right) using AVAudioSourceNode [2]. A composite generator optionally mixes binaural beats with background noise at configurable ratios.

Design Decision 1: Stereo Requirement

Binaural beats require stereo headphones to work—the effect depends on each ear receiving a different frequency. The app detects mono output routes and displays a warning, falling back to standard noise generation.

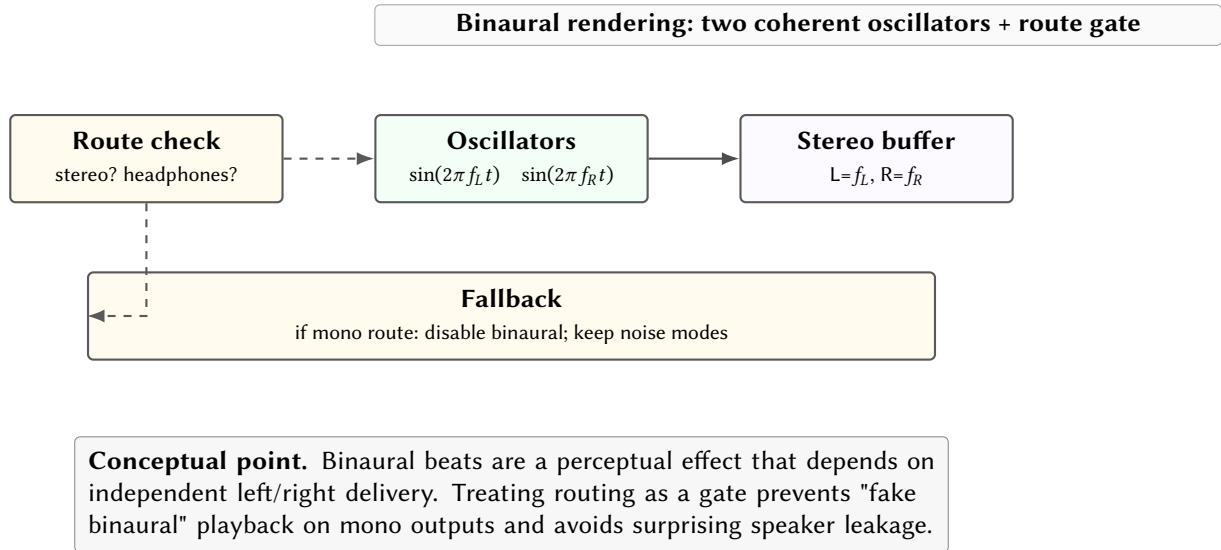


Figure 4. Binaural beats as a guarded stereo render path. A route gate ensures the effect only runs when the output path supports true stereo separation.

3 Playback State Machine

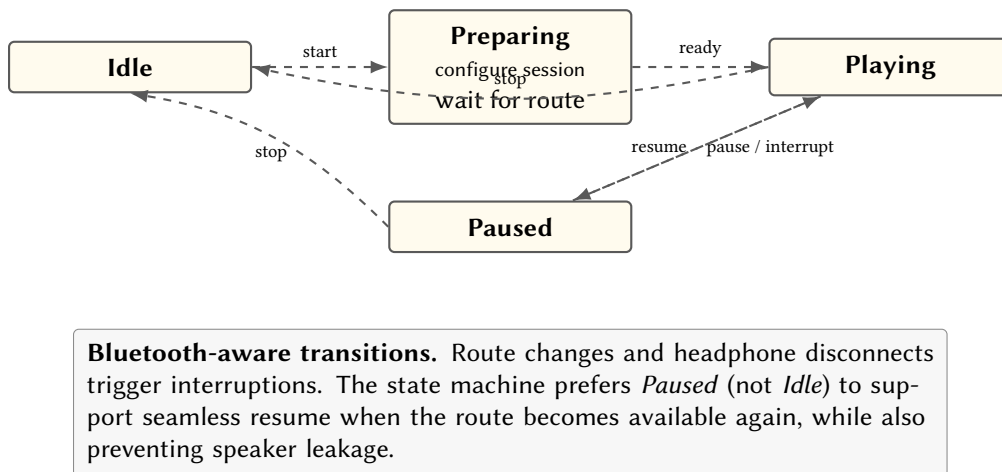


Figure 5. Playback state machine with explicit transitions for user actions and Bluetooth interruptions.

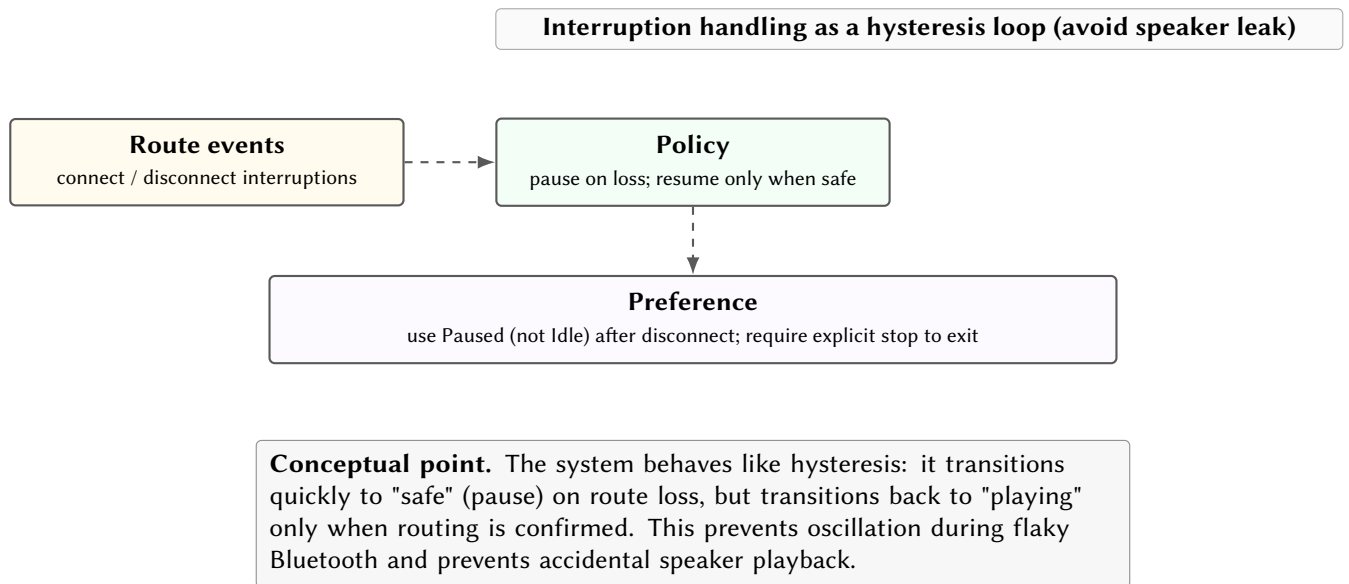


Figure 6. Bluetooth-aware playback policy as hysteresis. Fast pause on disconnect; conservative resume after route confirmation to prevent speaker leakage.

The `PlaybackManager` coordinates audio engine, session, and Bluetooth state through a four-state machine:

1. **Idle** — No audio session active
2. **Preparing** — Session configured, waiting for Bluetooth route confirmation
3. **Playing** — Audio engine running, source node generating samples
4. **Paused** — Engine paused (headphone disconnect or user action)

The session manager monitors Bluetooth route changes via `AVAudioSession.routeChangeNotification`. When headphones disconnect, playback automatically pauses to avoid audio leaking through the watch speaker.

4 Design Summary

Table 2. Key design decisions.

Decision	Rationale
Procedural synthesis	No storage overhead; infinite non-repeating audio
IIR brown noise	Single multiply-add per sample; minimal CPU on watch
<code>AVAudioSourceNode</code>	Real-time callback; no buffer pre-filling needed
Auto-pause on disconnect	Prevents speaker leak; respects sleep context
Digital Crown volume	Eyes-free interaction for sleep use case

Insight:
The Digital Crown is mapped to volume control via `WKInterfaceDevice.play(.c)` for haptic feedback, providing a physical volume knob without visual interaction— ideal for adjusting volume in the dark while falling asleep.

References

- [1] G. Oster, "Auditory beats in the brain," *Scientific American*, vol. 229, no. 4, pp. 94–102, 1973.
- [2] Apple, *AVAudioEngine: Audio processing graph*, 2024. [Online]. Available: <https://developer.apple.com/documentation/avfaudio/avaudioengine>

List of Figures

1 WatchNoise audio synthesis pipeline. The app is built around the real-time render callback; everything else exists to feed it parameters safely.	2
2 Brown noise generation as a minimal-state filter. One multiply-add plus one state variable shapes the spectrum toward low frequencies.	3
3 Noise blending as control-signal composition. The UI sets a target mix ratio; the audio thread applies a smoothed crossfade per sample (or per block) to avoid discontinuities.. . . .	3
4 Binaural beats as a guarded stereo render path. A route gate ensures the effect only runs when the output path supports true stereo separation.	4
5 Playback state machine with explicit transitions for user actions and Bluetooth interruptions.	4
6 Bluetooth-aware playback policy as hysteresis. Fast pause on disconnect; conservative resume after route confirmation to prevent speaker leakage.	5

List of Tables

1 Binaural beat presets.	4
2 Key design decisions.	5

List of Design Decisions & Rules

1	Design Rule: Procedural Generation Only	2
1	Stereo Requirement	4

Revision History

Version	Date	Changes
0.1.0	March 2026	Initial architecture report